Automation in Construction 182 (2026) 106756

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Early-stage architecture design assistance by LLMs and knowledge graphs

Danrui Li [a,*], Yichao Shi [b], Mathew Schwartz [a,c], Mubbasir Kapadia [a]

[a] Rutgers, the State University of New Jersey, 110 Frelinghuysen Rd, Piscataway, 08854, NJ, USA
[b] Georgia Institute of Technology, 631 Cherry St NW, Atlanta, 30332, GA, USA
[c] New Jersey Institute of Technology, University Heights, Newark, 07102, NJ, USA

ARTICLE INFO

Dataset link: https://github.com/danruili/ArchLogicRAG

Keywords:
Architecture design assistance
Large language model
Retrieval-augmented generation
Knowledge graph
Information retrieval

ABSTRACT

Early-stage architectural design relies heavily on precedent cases and domain knowledge, yet existing assistance methods struggle with the dominance of visual data and the linguistic diversity of design descriptions. In this paper, a retrieval-augmented generation framework with a custom knowledge graph tailored to architecture is proposed. The approach features: (1) a lightweight graph structure representing design logic; (2) a knowledge extraction pipeline for visual and textual data; and (3) aggregation and question answering methods that consolidate precedent knowledge for design support. Experiments show improved retrieval accuracy, more comprehensive precedent recommendations, and enhanced user experience, advancing precedent-based assistance for early design.

1. Introduction

Early-stage architectural design is an open-ended, iterative process characterized by uncertainty, creativity, and the need for rapid exploration of conceptual alternatives [1]. In early design sketching and before CAD models are created, designers frequently refer to precedent cases — built or proposed design projects — as valuable sources of inspirations for spatial strategies, stylistic cues, and functional arrangements [2,3]. These precedents typically include both visual documentation and textual narratives, providing a rich, multidimensional basis for idea generation. Efficient access to and effective use of such precedent cases are especially critical for novice architects [4] and architecture students [5], who often lack a broad repertoire of design references and benefit from external stimuli to initiate and develop early design concepts.

With the increasing availability of digital design archives, there is growing interest in using computational methods to support early-stage design through structured access to precedent cases. While early approaches were diverse [6,7], Large Language Models (LLMs) have gained attention for their strong multimodal understanding and reasoning capabilities in design disciplines [8], opening new possibilities for precedent-based design tools. Among LLM-involved methods, retrieval-augmented generation (RAG) has been widely studied: it helps LLMs improve their answers by pulling in relevant and up-to-date information from external sources at the time of the query [9,10]. Complementing this, knowledge graphs offer structured representations of domain-specific concepts and their relationships [11]. It reduces the information redundancy and gives the system a broader, more coherent understanding of the topic. While not inherently LLM-based, knowledge graphs can be integrated with LLM-based RAGs (RAG-KG) to enhance document preprocessing and reasoning coherence, particularly valuable for summarizing global-scale information [12].

However, early-stage architectural design presents domain-specific hurdles to apply RAG and knowledge graphs. Much of the relevant knowledge is embedded in visual media such as diagrams and photographs, which necessitates specialized techniques for extracting domain-specific content to improve retrieval quality. Another challenge lies in the domain's linguistic diversity. For example, the same idea can be expressed in different ways ("fenestration is sparse" versus "few openings"). Furthermore, design knowledge incorporates a wide variety of shapes and spatial configurations. These variations impose challenges to standardized knowledge graphs that rely on predefined concepts and relationships.

To address the challenges of visual data dominance and linguistic diversity in architectural design case data, this paper adopts a RAG-KG framework but extends it with a custom knowledge graph structure tailored to the architectural domain. The contributions are threefold:

- **Custom knowledge graph structure**: A lightweight knowledge graph framework tailored to early architectural design is proposed. Using sentence-level node representations, the framework captures design logic as correlational relationship tuples linking design strategies to design goals.

* Corresponding author.
E-mail address: danrui.li@rutgers.edu (D. Li).

https://doi.org/10.1016/j.autcon.2025.106756
Received 27 May 2025; Received in revised form 29 December 2025; Accepted 30 December 2025
Available online 10 January 2026

- **Multimodal knowledge extraction**: This paper proposes a robust pipeline that processes both visual and textual data from design cases and converts them into structured design logic tuples. Unlike prior work that focused only on extraction [13], the proposed approach explicitly generates graph-ready representations, establishing the foundation for knowledge graph construction.
- **Knowledge aggregation and utilization**: Building on [14], this paper introduces an aggregation method that consolidates knowledge across cases into the custom graph. A multi-step question answering pipeline that leverages the aggregated knowledge graph is developed as well enabling precedent-based design assistance as a practical tool for end users.

This paper demonstrates that the framework successfully utilize the design knowledge in unstructured visual data of the precedent case database. Compared to prior work, this paper leads to better accuracies in retrieval tasks, more related and comprehensive design case references in question answering, and better overall user experience. By solving the visual dominance and linguistic diversity challenges in precedent case data, this paper contributes to a more informative and compatible design assistance system in early-stage architecture design.

2. Related work

This section begins with a broad overview of computational design assistance in Section 2.1, positioning this paper within the category of knowledge-based and precedent-driven tools. Section 2.2 then examines how precedent-driven systems encode and organize design knowledge, contrasting early case-based methods, knowledge graphs, and LLM-based retrieval to motivate the proposed lightweight graph design. Finally, Section 2.3 surveys efforts to incorporate visual information into knowledge-based systems, outlining existing computer-vision approaches and multimodal models that inform the proposed design-centered knowledge extraction strategy.

2.1. Early-stage architecture design assistance

Architectural design, especially in early stages, is a creative and exploratory process where designers must consider a wide range of factors, from spatial organization and aesthetics to programmatic and environmental concerns, often without fully developed CAD models [15]. To support this phase, prior research has introduced tools aimed at enhancing either the efficiency of design workflows or the quality of design solutions. Some systems focus on improving human–computer interaction, such as sketch-to-CAD converters [16] and extended reality platforms for design education [17]. Others explore generative design models to assist early ideation [18].

Many existing approaches emphasize performance-based evaluation and optimization, using models to predict metrics like energy consumption [7,15], thermal comfort [19], walkability [20,21], and occupant safety [22]. These tools often depend on detailed geometric inputs such as 3D massing or BIM models.

While such methods are valuable for performance-driven refinement, early-stage design frequently integrates aesthetic, cultural, and conceptual factors that cannot be fully covered by quantification methods. For instance, even in tasks like energy optimization, the visual and spatial qualities of elements (*e.g.*, louver design) are deeply entangled with technical decisions.

As a result, knowledge-based and precedent-driven tools have emerged to provide richer, more flexible support, allowing designers to engage with reference cases and qualitative insights without being constrained by rigid evaluation criteria.

2.2. Knowledge representation in precedent-driven tools

For knowledge-based and precedent-driven tools, a critical factor in their effectiveness lies in how well the design knowledge and precedent cases can be represented and aggregated. Case-based reasoning (CBR) systems at early times [23], encoded cases using simple numerical attributes such as floors and area, where retrieval is based on matching these low-dimensional features [6]. While efficient, this representation neglects the rich, interrelated nature of design knowledge, reducing complex concepts to isolated variables. Later studies attempted to capture relation complexity by introducing attributes processed by deep-learning models [24]. But the relationships are implicitly learned by models, which prevents further utilization.

To better capture the complexity and the diversity of relationships inside design cases, knowledge graphs encode information as nodes (entities) and edges (relationships) [25]. These graphs typically follow a "subject–predicate–object" triplet format and can consolidate knowledge across many cases. However, this increased expressivity comes at the cost of representational rigidity. As researchers sought to generalize to broader domains, knowledge graphs grew increasingly complex: incorporating large vocabularies [10,26], using layered [27] or nested graph structures [28]. These rigid, complex schemas make knowledge graphs effective in structured rule-bound knowledge where precise matching is required (*e.g.* review contract documents [28] and academic publication [11]). In those cases, the value of a property can be matched lexically (location of project) or numerically (year of completion). While they can perform queries with complex filters by advanced query languages such as Cypher [29], they struggle to accommodate the open-vocabulary, context-sensitive, and often ambiguous information in early-stage architectural design.

In contrast, recent LLM-based retrieval-augmented generations (RAGs) avoid schemas altogether. They operate in a continuous semantic space, comparing text embeddings to retrieve relevant information based on meaning rather than structure. This enables them to navigate stylistically diverse or loosely defined language without manual ontology design [9,30]. Later studies further adapted general LLMs to architecture and construction scenarios by in-domain [31] or out-of-domain [32] fine-tuning. However, these methods represent a different extreme: by discarding explicit structure, they forgo the ability to model fine-grained, interpretable relationships between concepts, which is an important aspect for design reasoning and customization. GraphRAG [12] strikes a balance between these two paradigms by using LLMs to summarize high-level information from knowledge graphs, which compensates the lack of complex reasoning on rigid graph schema. However, its graph still follows the atomic triplet format, which makes it widely compatible but less ideal for design knowledge.

This paper also falls into the spectrum between traditional knowledge graph and RAG, but is equipped with an even more minimal ontology design than GraphRAG [12]. Inspired by sentence-level node representations in commonsense reasoning [33,34], this paper develop a lightweight knowledge graph framework tailored to early architectural design: Nodes capture high-level design logic, such as strategies or goals, in natural language, while edges encode their relationships. LLMs support the extraction and querying process, allowing us to minimize rigid ontology design while retaining interpretable, structured reasoning.

2.3. Integrating visual understanding

Design research has a long tradition of formal approaches to visual and spatial reasoning. Early work on shape grammars [35,36] provided a rule-based formalism to describe and generate design variations by systematically transforming geometric configurations. Similarly, Alexander's notion of pattern languages [37] emphasized recurrent spatial organizations and design heuristics, which influenced subsequent computational frameworks for encoding design knowledge.

In the AEC domain, room adjacency graphs [38] and related spatial graph models [39] have been widely employed to analyze floor plan layouts, circulation, and programmatic relationships. While these approaches highlight the importance of structured representations for visual–spatial reasoning, they also have notable limitations: they often depend on manually defined rules or structured BIM data, lack scalability for large and heterogeneous datasets, and are not readily applicable to unstructured inputs such as natural language descriptions, photos, or diagram images.

To deal with this issue, [40] proposed a deep learning framework to update knowledge graphs using video input. [40,41] used object detection methods to identify physical objects from construction site cameras, where the knowledge graph represents spatial interactions between objects. While object detections are sufficient and robust when a small set of objects and relations is considered, they are difficult to extract diverse semantic information such as materials, spatial configuration, and styles without developing specific expert models.

These constraints motivate the integration of multi-modal large language models, which can automatically extract diverse, semantic, and relational cues from visual data [9,13]. By using [13] to extract information from visual data, this paper further develop a pipeline that convert the extracted information to design logic tuples that can be integrated into knowledge graphs.

3. Method

The main functionality of the framework is shown on the right of Fig. 1. Given a set of architecture design cases that consists of both textual descriptions and images in a database, the proposed system provides a conversational interface that allows users to "chat" with the system to search design cases from the database, or ask general design questions about it, such as *"how to maximize daylight usage?"*. The system then provides a response with detailed analysis and providing design cases as references.

Fig. 1 illustrates the technical pipeline, which is a RAG system with aggregated knowledge generated by a custom knowledge graph. Starting from unstructured asset files, Section 3.1 describes how the proposed approach uses LLMs to extract design descriptions and design logic from the raw materials. Section 3.2 showcases how the design logic is aggregated across all design cases by the knowledge graph. Section 3.3 details how the extraction can support retrieving related design cases by user instructions, which lays a foundation to the core question answering pipeline in Section 3.4. In Section 3.5, how the framework facilitates a conversational interface is explained. The last section describes the evaluation method for the pipeline.

3.1. Extracting design logic from raw assets

The framework starts with extracting design logic from the texts and images for each design case in the database. The following text first describes how the design logic is represented, and subsequently how it is extracted.

Design logic can be represented either as plain textual statements within RAG pipelines [30], or as relationships between low-level entities in knowledge graphs [41]. This research concurrently utilizes both plain textual statements and a sentence-based graphical representation. The proposed representation is formulated as correlational relationships linking design strategies to specific design goals. This paper defines "strategy" as physical appearance or spatial arrangements of the design cases, and "goal" as their intended functional, social, environmental, or aesthetic impacts. For example, the strategy *'usage of large windows'* serves the goal *'maximize the indoor lighting'*. In this way, the knowledge in one design case can be represented as multiple strategy–goal relationships, where one strategy may be mapped to multiple goals and vice versa (see Table 1 for more demonstrations). In database, such relationships are stored as graph edges, where origin nodes are strategies and target nodes are goals (see the right side of Fig. 1). This approach aligns conceptually with the "eventuality" relations described in [33], as well as the "if-then" relations presented in [34]. In the proposed approach, textual statements are used for retrieval (Section 3.3), and strategy–goal relationships are used for aggregation (Section 3.2).

Based on prior practice in [13], this paper uses `gpt-4o-2024-08-06` (abbreviated as `GPT-4o`), a large language model with multimodal understanding, to extract design logic from the texts and images. In all other components and experiments in this paper, the same LLM is used unless specified. Fig. 1 shows the extraction process: Each asset (either text description or image) is sent to the LLM with manually-designed system prompts, where two pipelines are used to extract design logic in different approaches. The first pipeline (referred to as *Domain-tailored analysis* in Fig. 1) uses the same method and system prompt as [13], which directly analyze the architecture design in 6 predefined aspects (form, style, material, emotional feeling, passive design, and general highlights). The aspect selection follows the practice of [13], which originated from a user survey that studied users' focuses in case studies, covering both architecture students and professional architects. In the *Reformat* stage, the analysis is formatted as strategy–goal text tuples, stored in JSON data format (see LLM prompts in Section 6.2.1).

The second pipeline (referred to as *Domain-tailored extraction* in Fig. 1) is multi-step; (1) it starts by instructing the LLM to describe the image in an objective and detailed way (see LLM prompts in Section 6.2.1). The descriptions are later used as one source of the text statements in retrieval tasks. (2) after the same reformat process, the pipeline 'encourages' the LLM to complement its answer multiple times (called "gleaning" in [12]). Furthermore, if an image is being processed, the pipeline feeds the LLM with the raw textual description from the database, instructing it to add more strategy–goal tuples, if any statements in the raw description are found in the image. In this way, the pipeline maximizes the design logic extraction, which benefit the downstream tasks.

A small-scale internal spot-check evaluation is conducted to assess the correctness of the design logic tuples. One researcher evaluated 100 randomly sampled tuples from the generated results. Of these, 96 were labeled as correct by human, 4 were uncertain to judge, and none were labeled as incorrect, indicating a high degree of correctness. This work does not explicitly address incorrect tuples. However, given the high accuracy observed in the spot-check evaluation, the current approach provides a sufficiently reliable foundation for this research.

3.2. Aggregating knowledge across design cases

As the design logic for each design case has been extracted in 3.1, now the design logic is aggregated across all cases. It enables a global understanding between design strategies and goals. For example, it is possible to investigate which strategies can be applied to a specific design goal, or which design cases support similar design goals, across all design cases in the database.

Due to the diversity of design cases and the stochasticity of LLM extractions, similar nodes (strategies or goals) may differ in their text forms. For example, *'use large windows'* and *'the usage of expansive glazing'* express almost the same idea but are lexically different. To group nodes across design cases by what the words really mean, the text embeddings of all nodes are computed by a pretrained model (`text-embedding-3-large` by OpenAI), then project the embeddings to a 10-dimensional space by Uniform Manifold Approximation and Projection (UMAP) [42], where the dimension parameter follows the practice in [14]. Finally, a Gaussian Mixture Model (GMM) implemented by [43] is used to group the strategy nodes and goal nodes separately by the cosine similarity of their projections. Since GMM groups all nodes into a specified number of clusters, the Bayesian

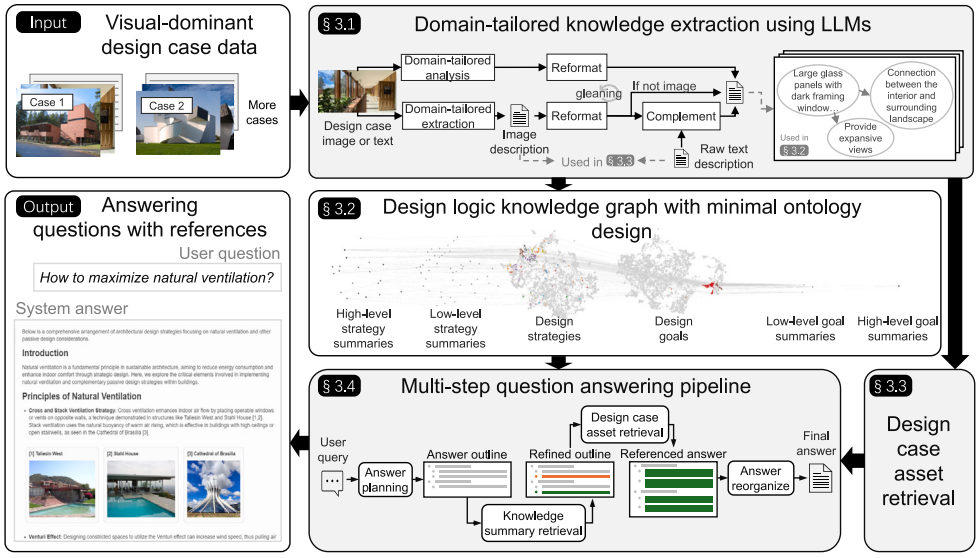
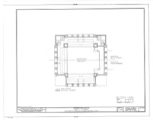


Fig. 1. Framework overview.
The proposed approach extracts a design-logic knowledge graph from unstructured design case assets, enabling question answering with design case references.

Table 1
Extracted design strategies and goals from design case images.

Source	Strategy	Goal
	The building features a prominent vertical sign with curved edges on the corner.	Enhance visibility in the urban environment.
	Inclined roof planes supported by visible wooden beams with translucent panels.	Allow diffused light to enter the interior space.
	Vertical and horizontal lines are sharply defined across the building.	Create a dynamic visual rhythm.
	The perimeter walls are evenly punctuated by window openings.	Provide visual connection to the exterior from within the building.
		Ensure natural light penetration within the building.

Information Criterion is applied to select the best number of clusters (searching from 30 to 240 in this work).

After grouping all nodes that were extracted from design case materials, the LLM is called to write summaries for each group, forming summary nodes that aggregate knowledge across different cases. To provide more sufficient context, the node content is represented in strategy–goal pairs. For example, when summarizing strategies, the LLM is provided with both the content of strategies and the content of their corresponding goals. After getting the summary nodes, the summary nodes are further clustered (separating strategies and goals) and higher-level summaries are generated for them in the same way, thus creating a bi-level knowledge structure.

Fig. 2 showcases how a high-level strategy cluster emerges from summarizing low-level strategies that are directly extracted from raw materials. Starting from 257 related descriptions extracted from raw materials in `Wikiarch` database on the figure left, the pipeline first generates six low-level strategy clusters, where summary contents are not shown here. Then, a high-level strategy cluster *"Material Strategies and Their Roles in Architectural Design"* is summarized from the six clusters, with the beginning of its summary shown below its title. The references to design cases are displayed in blue texts.

Due to the limitation of LLM capabilities, the node clustering results are not always aligned to human judgment. For example, in Fig. 2, human experts may consider merging the low-level clusters *"Use of Materials in Architectural Design"* and *"Material Usage in Architectural Design"*, though the former one contains more design strategies that involve stone materials, while the latter involves more diverse uses. However, this discrepancy does not harm the approach of this paper, as this paper does not retrieve only one summary item but a combination of multiple summaries as described in Section 3.5.

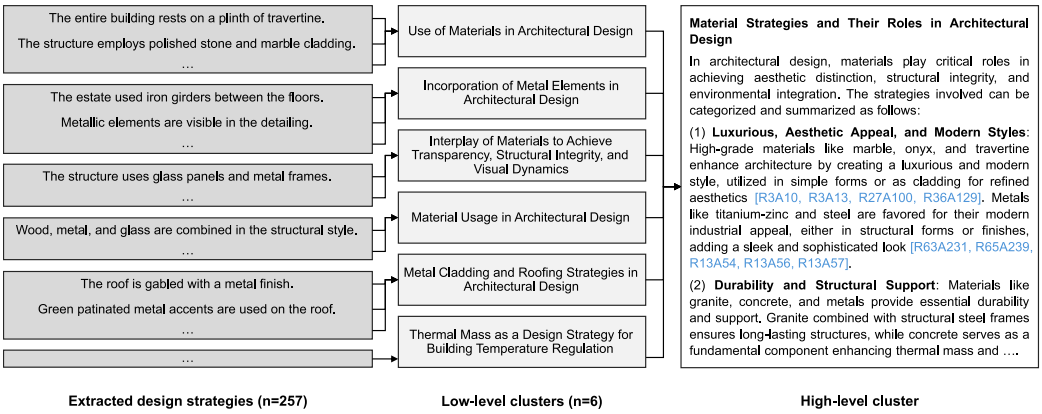


Fig. 2. Example of design logic summarization.

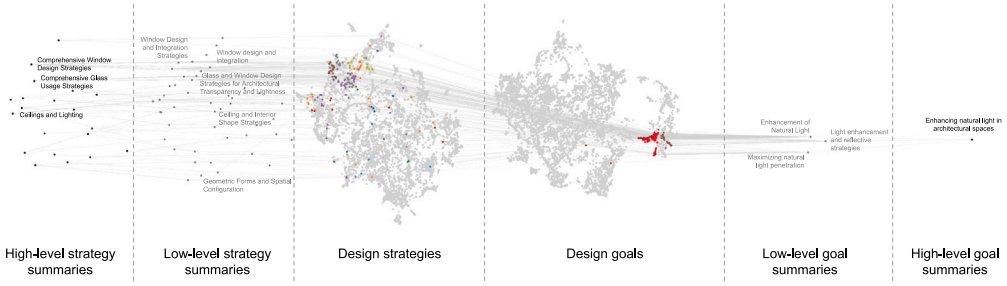


Fig. 3. UMAP projections showing how high-level design goals (e.g., "Enhancing natural light") relate to supporting strategies.

The summarization process compresses the knowledge by removing duplicates across different cases, providing more comprehensive and diverse information than original nodes, thus improving the output detailed in 3.5.

Based on the extracted knowledge, Fig. 3 shows that the proposed bottom-up approach can aggregate a reasonable knowledge structure across all design cases. From 7558 strategy–goal pairs that were extracted from 70 cases in `WikiArch` dataset, 185 strategy summaries (125 low-level summaries and 30 high-level ones) and the same number of goal summaries were generated. The design strategies and goals, together with their summaries, are visualized using UMAP projections of their text embeddings. Related strategies and goals are grouped by GMM-based clustering. As illustrated, starting from a high-level design goal such as *"Enhancing natural light in architectural spaces"* (black dot on right side), its supporting strategies such as *"window design"* or *"glass usage"* can be backtracked (left side), which aligns to human judgment. These backtracking paths are shown as light gray lines in the middle of the figure.

The knowledge graph is constructed and managed using LlamaIndex [44], leveraging its native document and vector store to facilitate code reproducibility. While this setup ensures ease of implementation, this paper recommends adopting more efficient database solutions for production-level applications to improve scalability and performance.

3.3. Retrieving design cases by user query

The extracted design logic from Section 3.1 is used to enable design case retrieval capability, which is a corner stone of question answering. It returns a list of design cases ranked by the relevance to user instructions such as *"give me some examples of roofs"*. The retrieval is achieved by the Reciprocal Rank Fusion (RRF) algorithm [45], which fuses the ranking results from an ensemble of retrieval methods. The retrieval methods used in this work are as follows:

Image description and raw text matching: This method builds upon common practice of RAG systems in prior work [9,30]. Specifically, during a pre-processing stage, image descriptions are chunked in Section 3.1 by paragraphs. Raw textual content from the dataset is also segmented using a fixed length of 280 characters with a sliding window of 200 characters to ensure contextual overlap.

All text segments, including those from image descriptions and raw text, are then converted into embeddings using the same language model described in Section 3.2. During retrieval, the cosine similarity between each text segment's embedding and the embedding of the user query are computed. These segments are then ranked based on their similarity scores, which is also named as dense retrieval [46].

The proposed method diverges from conventional RAG by introducing an additional aggregation step. Each design case is assigned with the rank of its highest-ranked associated text segment, facilitating a case-level retrieval rather than at the segment level.

Analysis statement matching: The first retrieval method is altered to rank the similarity between domain-tailored analysis (see Section 3.1) and the user query. Chunking is not applied here since the statements are one sentence in length.

Image embedding matching: The first method is altered by replacing the embedding model to a multimodal one [47]. Then, the cosine similarity between the embedding of all image assets and that of user query is compared. The rank of design case is decided by its highest-ranked images.

Given a user query, each retrieval method returns a ranked list of design cases. The RRF algorithm fuses their rankings by summing up the inversion of the rank indices. Specifically, for a design case d and an ensemble of rankings R, the final relevance score for d is:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{r(d) + c} \tag{1}$$

where the hyperparameter $c = 10$ in this paper.

3.4. Utilizing aggregated knowledge in question answering

The aggregated knowledge described in Section 3.2 enables the primary capability of the framework. When users ask general architecture

design questions, the framework provides a detailed answer with design case references, where the internal knowledge of the LLM is augmented with aggregated knowledge from the database.

As shown in Fig. 1, the proposed method begins by prompting the LLM to draft an answer outline, forming a two-level structure consisting of sections and list items. Then, related summary nodes (created in Section 3.2) are retrieved from the database, and the LLM is instructed to refine the outline using the retrieved summaries. Next, the method iterates over all list items in the improved outline, retrieving related documents for each item, and using the LLM to complete the detailed answer based on the asset retrieval results. The asset retrieval at this step follows the same method in Section 3.3, except that the rankings are fused based on design case assets for finer-grained retrieval, rather than design cases. In this way, this method enables a fine-grained retrieval that locates the most relevant assets (images or text), of which text descriptions are fed to the LLM to integrate the assets into list items. Finally, the answers from all list items are re-organized by the LLM, merging into one final answer.

3.5. Integration by LLM agents

To provide a friendly user experience, a conversational user interface that integrates both capabilities above is built. By typing user instructions into the text field, the user interface facilitates a chatbot-like experience in design assistance (see Fig. 1), backed by a custom-designed LLM agent pipeline.

This pipeline operates as follows: given a user instruction and chat history, an LLM-based *Router Agent* will first determines the appropriate task-specific agent to handle the request, either *Design Case Search Agent* or *General QA agent*. Then, the *Router Agent* reformats the user intent into function arguments of downstream agent and calls the agent. The selected agent processes the input and returns a response to the user. This paper adopts this router-based system structure to enable flexible integration of more task-specific agents and tools in future work.

The *General QA Agent* corresponds to the implementation described in Section 3.4, while the *Design Case Search Agent* builds on the method in Section 3.3, with an additional summarization step performed by an LLM. For detailed system prompts used in the LLM components, please refer to the appendix.

The conversational system is implemented using the Flask web framework [48]. It connects an HTML-based graphical user interface to a Python backend, where the LLM agent pipeline is implemented using OpenAI's native API toolkit. The graphical user interface is adapted from the code base in [49].

3.6. Evaluation

The proposed framework is evaluated in a three-fold manner. First, the quality of design case retrieval is measured, as it constitutes the cornerstone of the question-answering capability. Performance is evaluated against prior retrieval methods using the quantitative benchmark specified in Section 3.6.2. Next, Section 3.6.3 assesses whether the retrieved design cases are effectively and comprehensively utilized during answer generation, where performance is compared against two RAG approaches. Finally, Section 3.6.4 compares the proposed framework with an off-the-shelf LLM (`GPT-4o`) through overall user experience measures and interviews.

3.6.1. Dataset

To maximize the reproducibility of this work, Wikipedia was selected as the primary dataset (referred to as `WikiArch`) for evaluation. Wikipedia pages related to architectural design from the early 1900s to the 1960s were used, with entries that lacked high-quality images or sufficient textual descriptions manually excluded. Ultimately, 70 architectural cases were selected for the dataset, covering a wide range of functions and styles. For each case, the complete textual content was retrieved directly from the corresponding Wikipedia page using `Wikipedia API` [50], and up to five images were manually selected to serve as visual assets. To further evaluate whether the proposed method can generalize to cases that may not appear in the training data, an additional dataset (referred to as `Gallery`) from [13] is used to complement the evaluation. It contains 54 cases, mostly newly built art galleries or museums.

3.6.2. Retrieval quality

A retrieval benchmark is constructed by correlating 65 user queries with design cases in the `WikiArch` dataset based on 16 human-annotation survey responses. The queries are manually designed to balance between various topics (material, form, emotional feeling, etc.) disclosed in a prior work [13]. Most annotators were architecture students, selected to reflect typical user demands when using the framework.

Next, the framework is tested using these queries, and the retrieved results are compared with the human-labeled data. Following prior practice [51], an adjusted version of Normalized Discounted Cumulative Gain (NDCG) is adopted as the evaluation metric. Given a user query q and an evaluation scope K, NDCG@K measures how close the produced ranking is to the perfect ranking for the top K items. Specifically, Discounted Cumulative Gain (DCG), which quantifies ranking quality under the produced ordering, is first computed as:

$$\mathrm{DCG@}K = \sum_{i=1}^{K} \frac{rel_i}{\log_2(i+1)} \tag{2}$$

where rel_i is modified from the original formulation to denote the importance score of the item ranked at the ith position in the retrieval results.

For a given query q and item t, the importance score $\mathrm{I}(q,t)$ is aggregated from a subset of human annotators who label t as relevant to q. For each annotator h in the subset $H(q,t)$, the importance *"allocated"* to item t is computed as 1 divided by the total number of items labeled as relevant to q by annotator h, denoted as $N(q,h)$. The importance score for t is then obtained by summing the allocated importance across all annotators.

$$\mathrm{I}(q,t) = \sum_{h \in H(q,t)} \frac{1}{N(q,h)} \tag{3}$$

Finally, $\mathrm{I}(q,t)$ is divided by the average non-zero importance scores for each query, so that the average non-zero score is 1.

Next, Ideal Discounted Cumulative Gain (IDCG) is computed to quantify the quality of the ground-truth (perfect) ranking. IDCG follows the same formula as Eq. (2), except that the ranking is replaced by the ground truth, where all related items are ranked in front of all other items.

NDCG@K is the ratio between the two. When the value is closer to 1, it achieves better performance.

$$\mathrm{NDCG@}K = \frac{\mathrm{DCG@}K}{\mathrm{IDCG@}K} \tag{4}$$

Since the dataset is relatively small, an adjusted variant is applied to discount retrieval gains that can arise from random selection. Specifically, aNDCG@K is defined as the additional "gain" achieved by the proposed method over a random ranking, divided by the additional "gain" achieved by the perfect ranking over a random ranking. Randomized Discounted Cumulative Gain (RDCG) is computed in the same manner as Eq. (2), except that the produced ranking is replaced with a randomized ranking. The resulting aNDCG is then calculated as:

$$\mathrm{aNDCG@}K = \frac{\mathrm{DCG@}K - \mathrm{RDCG@}K}{\mathrm{IDCG@}K - \mathrm{RDCG@}K} \tag{5}$$

All aNDCG@K values are averaged across user queries to obtain an overall aNDCG@K under a sampled random ranking. To mitigate stochasticity from random-ranking sampling, bootstrapping is further applied on the random-ranking results ($n = 1000$). Finally, the mean

and standard error of aNDCG values across different top-K settings are reported in the Results section (Table 2). The proposed method is compared against a naive baseline, a prior approach [13], and two ablated variants introduced here. The naive baseline simply chunks the original texts without the additional text extraction described in Section 3.1, and uses these text chunks to retrieve related design cases. The chunking procedure follows Section 3.3. In the two ablated variants, the use of extracted texts (image descriptions and analysis statements) and image-embedding matching are removed, respectively. The results are reported later in Table 2.

3.6.3. Reference quality

While the prior experiment evaluates how well the system can retrieve related design cases based on user queries, it does not assess whether the retrieved results can be well utilized in generated answers. This section examines the quality of design case utilization from two perspectives: fitness and aggregated diversity.

Fitness refers to how well the referenced design cases are integrated into the answer content generated by the LLM-based framework. This is assessed through a user rating study. Ten participants were recruited from five groups: architectural researchers, professional architects, undergraduate architecture students, non-architecture researchers, and non-architecture students. This diverse participant pool allowed us to include both domain experts and non-experts, thus capturing a broad range of perspectives on the usefulness of the design references.

Each participant received a user survey consisting of 40 randomly sampled sections of generated answers. Each section includes a short description of a design concept (*e.g.*, orientation and shading, or community and social spaces) followed by one or more architectural design cases accompanied by images. For each design case reference, participants were instructed to provide binary true/false responses to the following two statements:

- **Case Fitness (CF)** The design case reference can serve as inspiration to support the analysis of the design concept. For example, *Farnsworth House* can be considered inspirational for the design concept *"enhance sunlight usage"*.
- **Image Fitness (IF)** The reference image appropriately illustrates the corresponding design case description. For example, if the description focuses on the exterior landscape of *Fallingwater* but the image shows an interior view, then the statement is marked false.

Survey responses were aggregated as follows: for each design case in each answer section, the proportion of "True" responses was computed for both CF and IF across all raters. These proportions were then averaged across all design case references to obtain the overall mean probabilities, which were used as the fitness metrics.

To enable comparisons with prior work, answers from the proposed method are evaluated alongside two baselines: naive RAG and GraphRAG [12]. The Nano-GraphRAG implementation [52] is used for both baselines. Across all three methods, answers are generated using the same question list in Section 4.2. For a fair comparison, a retrieval budget of 60,000 tokens per question is enforced for all methods, measured on the retrieved text prior to answer generation; any retrieved content exceeding this limit is trimmed. In addition, the same LLM used in this work is also used for both baselines. Furthermore, since naive RAG and GraphRAG does not have visual understanding capabilities, for each design case reference in their results, a random image from the corresponding design case is attached in the answer section. In this way, the answer sections from all three methods look visually similar to participants.

Aggregate diversity, on the other hand, describes the extent to which design cases from the dataset are utilized across all answers [53]. High aggregate diversity promotes long-tail utilization of the database, preventing the system from repeatedly drawing on a small, popular subset of design cases across varied scenarios, and thereby sustaining user engagement over time. This concept is operationalized using two metrics:

- **Case Coverage (CC)** Analogous to catalog coverage [54], it is defined as the proportion of distinct design cases cited at least once across all answers. Let C be the set of all design cases in the database and $U(c)$ be the number of answers that directly cite case $c \in C$. Then CC is calculated as:

$$\mathrm{CC} = \frac{|\{\, c \in C \mid U(c) > 0 \,\}|}{|C|}, \tag{6}$$

- **Case Dispersion (CD)** Analogous to distributional coverage [55], it is defined as the Shannon entropy of case usage frequencies. Let $p(c)$ be the proportion of citations assigned to case c. Case Dispersion is defined as:

$$\mathrm{CD} = -\sum_{c:U(c)>0} p(c) \log p(c), \tag{7}$$

CC captures the breadth of database leverage, while CD captures how evenly that leverage is distributed, penalizing concentration of citations in a few cases. Both metrics jointly form *Aggregate Diversity*, which quantifies both coverage and evenness of use.

3.6.4. Overall assessment

Overall question-answering experience is evaluated through a voting survey and interviews. Fifteen participants were recruited with a similar knowledge background to that described in Section 3.6.3. Prior to the evaluation, participants were asked to formulate three questions each based solely on their initial impressions of the user interface, without any prior use of the developed search system. This process yielded 75 questions, from which 14 were selected after removing duplicates and ambiguous entries.

Each participant was required to complete a voting survey followed by a 10-minute interview. In the voting survey, participants evaluated 14 pairs of answers to the same question, based on four criteria—comprehensiveness, diversity, empowerment, and directness—adopted from [12]. For each pair of answers, participants voted which answer was better for each criterion. Each pair consisted of one answer generated by directly prompting `GPT-4o` and one produced by the proposed method; both were presented in a consistent format, displayed anonymously, and ordered randomly. To further assess the impact of the proposed approach on textual quality, 50% of the generated answers were randomly selected and their associated images were removed (marked as the "text-only condition" in Section 4.3), enabling a direct head-to-head comparison with the `GPT-4o` outputs in text-only form. Voting results were converted into head-to-head win rates for each pair of methods by computing the proportion of answers in which one method was judged better than the other.

Following the voting survey, participants were interviewed to gather deeper insights into their satisfaction with the answer quality and suggestions for improvement. Interviews also explored their views on the system's potential usefulness in their respective fields and its possible impact on their learning and professional activities.

4. Results

This section first reports the quantitative retrieval benchmarks introduced earlier. Next, the quality of question answers is presented through qualitative examples and quantitative fitness and diversity metrics (Section 4.2). Overall experience is then evaluated using user ratings and interviews (Section 4.3). Finally, Section 4.4 explores how the method can be adapted to specific architectural design domains.

Table 2
Retrieval benchmark result.

(a) WikiArch dataset			
Model	aNDCG @ 5	aNDCG @ 10	aNDCG @ 20
Raw text chunking	0.10 ± 0.03	0.12 ± 0.02	0.15 ± 0.02
ArchSeek [13]	0.29 ± 0.04	0.29 ± 0.03	0.32 ± 0.03
Ours	0.32 ± 0.04	0.34 ± 0.03	0.35 ± 0.03
- No extracted desc.	0.25 ± 0.03	0.26 ± 0.03	0.27 ± 0.03
- No image emb.	0.31 ± 0.04	0.31 ± 0.03	0.34 ± 0.03
(b) Gallery dataset			
Model	aNDCG @ 5	aNDCG @ 10	aNDCG @ 20
Raw text chunking	0.00 ± 0.02	0.01 ± 0.02	0.01 ± 0.02
ArchSeek [13]	0.21 ± 0.04	0.25 ± 0.04	0.27 ± 0.04
Ours	0.23 ± 0.04	0.26 ± 0.04	0.29 ± 0.04
- No extracted desc.	0.16 ± 0.03	0.18 ± 0.03	0.21 ± 0.04
- No image emb.	0.14 ± 0.03	0.19 ± 0.04	0.21 ± 0.04

4.1. Design case retrieval

As the foundation to the downstream pipeline, the quality of knowledge extraction is demonstrated in the retrieval benchmark shown in Table 2. In this table, Adjusted NDCGs (mean ± standard deviation) are shown for WikiArch and Gallery dataset. A value of 0 represents the retrieval accuracy is equivalent to random selection, while a value of 1 represents a perfect match to human judgment. Metrics are shown in blue when they are significantly greater (pairwise student t-test, $P<0.05$) than prior work. The ablated variants are shown in the last two rows: "no extracted desc." denotes the retrieval method without using extracted image descriptions, and "no image emb." denotes the method without using image embeddings.

The method in this paper gains advantage over both the dense retrieval of raw text chunks and the prior work, demonstrating the enhancement from the fusion of rich extracted information. First, solely relying on raw text chunks achieves the lowest performance in the table. In particular, the raw text chunk method has no significant advantage over random selection in the Gallery dataset, suggesting texts from architecture project sites (which is a common source of new design cases) provides much less knowledge than those from Wikipedia. Second, the method without additional extracted texts ("no extracted desc." row) remains inferior to the proposed method, especially in cases where the raw texts provide insufficient information (e.g., the Gallery dataset).

How the retrieval quality varies with user queries is further analyzed. Fig. 4 shows a heatmap breakdown of performance for each query. For each user query (in x-axis) and each method choice (in y-axis), the adjusted NDCGs averaged from different top Ks (5, 10, and 20) in WikiArch dataset are shown in monochrome. Dark green denotes 1 (perfect retrieval) and white denotes 0 (equivalent to random retrieval). For visual clarity, user query content is displayed for every other entry.

The proposed framework performs well when users try to search specific architectural elements such as *"wood interior"*, where it achieves almost 100% alignment to human labeling. The retrieval performance decreases as it encounters more subjective queries (*"a sense of calm"*) or more complex reasoning (*"inspired by local culture"*). While retrieval quality can still be improved, the proposed method yields better retrieval results than the pure-text baseline (Raw text chunking in Table 2 and Fig. 4), providing an effective foundation for the subsequent pipeline.

Inter-annotator agreement was also analyzed, revealing substantial variability across annotators. For a given query, two randomly selected annotators shared approximately 21% of their labeled design cases. It should be noted that substantial diversity does not necessarily lead to retrieval failures. As shown in Table 2, the proposed method captures these preferences *in aggregate*. Rather than suggesting retrieval failure for the whole population, large diversity showcases the demand for *personalized* retrieval, which shed light on potential future work.

Table 3
Design case reference quality across methods. Four metrics are reported: Case Fitness (CF), Image Fitness (IF), Case Coverage (CC), and Case Dispersion (CD). Highest metrics are shown in blue.

Method	Number of		Fitness		Diversity	
	Refs.	Rated Refs.	CF(%)↑	IF(%)↑	CC(%)↑	CD↑
Naive RAG	120	53	55.2	46.7	57.1	4.95
GraphRAG	82	56	56.5	45.7	27.1	3.89
Ours	182	60	61.2	51.5	87.1	5.43

4.2. Question answering with concrete references

Resting on the design-case-referenced knowledge structure above, the proposed method augments the LLM with references from specific databases. In Fig. 5, user questions are shown as the titles in both subfigures, where answers are provided below in a unified visualization format. Compared to GPT-4o result (left), the result of the proposed method (right) provides richer information with concrete design case references. Due to page limit, both answers are clipped in the figures. The proposed method also inherits the internal knowledge structure of the LLM, as its result shows a similar structure as that of GPT-4o. Based on this, the proposed method further links the analysis with concrete examples. It not only attaches related case images after each paragraph, but also integrates the case-specific analysis with a high-level knowledge structure, providing a more intuitive and deeper understanding of design concepts for users.

Although GPT-4o has a larger knowledge base than the dataset used here, hallucinations were observed for some less-common queries. As shown in Fig. 6, given the user query *"give me some architecture projects where hexagonal tiles are used on facade"*, the answers of GPT-4o are firstly generated. Then the returned cases are manually searched online, where the researchers manually verified if the design cases aligned with the user query. The answer of GPT-4o is partially shown on the left-most column of the figure, only showing the project names and matching content segments. The architecture images are partially masked to comply with Fair Use Policy.

Two misaligned design cases were observed among the top four retrieved results: there are actually no hexagonal tiles in *The Hive* and *Maggie's Center in Manchester*, where GPT-4o claims that they have hexagonal tiles. On the contrary, the proposed method shows more robustness on hallucinations (right of Fig. 6). It returns results based on the availability of data without fabricating nonexistent facts.

Table 3 shows that the proposed method achieves higher design-case reference quality than the two baseline methods. As indicated by the *Case Coverage* and *Case Dispersion* metrics, the proposed method cites more diverse references than the baselines. This increased diversity does not come at the expense of fitness: *Case Fitness* and *Image Fitness* indicate that fitness remains comparable to the other methods.

Meanwhile, GraphRAG is observed to use less-diverse references than naive RAG. This disadvantage is further visualized using the cumulative distribution function (CDF) of case reference frequencies, sorted in descending order by the number of times each case is cited (Fig. 7). On the horizontal axis, N denotes the rank position among the most frequently cited cases, and the vertical axis shows the cumulative proportion of all references accounted for by the top N cases. Flatter curves represent more diverse references.

Low aggregate diversity is observed in GraphRAG answers: the steeper curve (orange) indicates that a small subset of cases accounts for a large share of references. Among all generated answers with different questions, over 80% of all GraphRAG references originates from just 10 cases, performing even worse than naive RAG (green). In contrast, a flatter curve (ours in blue) suggests that references are more evenly spread across the database, corresponding to higher Case Dispersion (CD) and broader Case Coverage (CC). Answers from GPT-4o are also included as a reference, as its broader knowledge base can be treated as an upper bound on diversity (flattest curve).

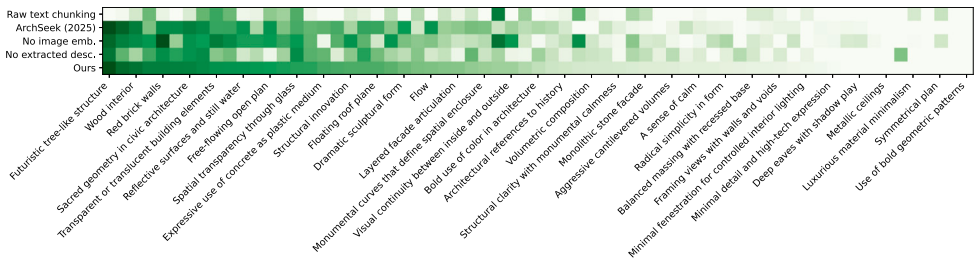
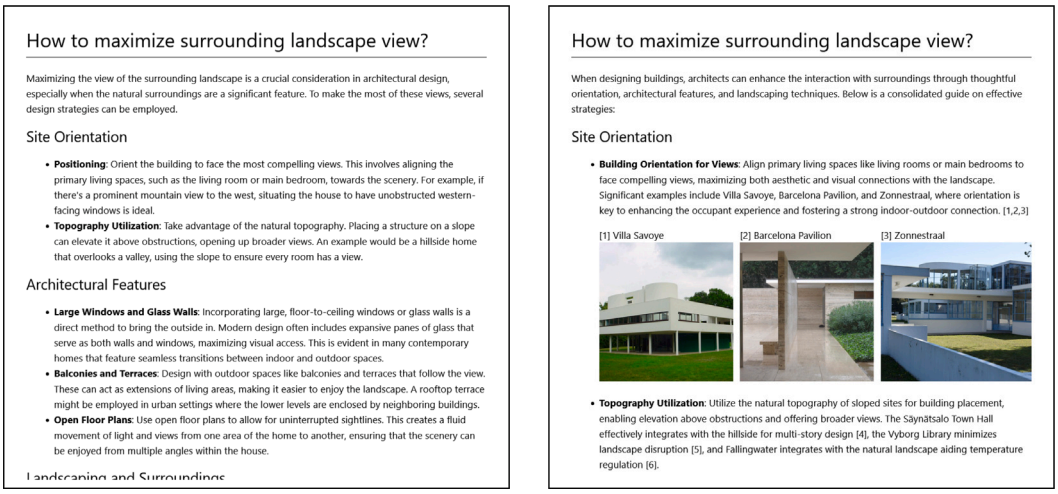


Fig. 4. Retrieval performance breakdown.



(a) GPT-4o. (b) Proposed method.

Fig. 5. QA example from GPT-4o and proposed method.

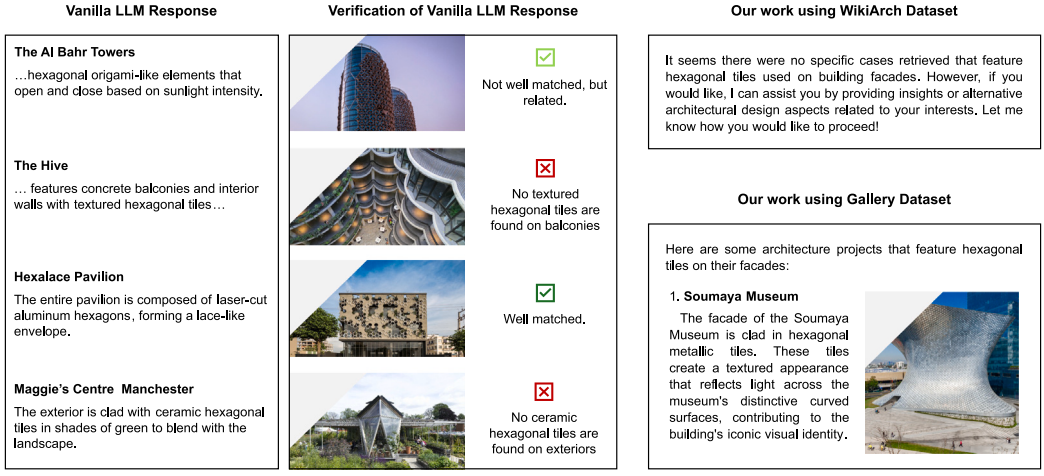


Fig. 6. Hallucinations in GPT-4o and the proposed method.

This result is attributed to the knowledge graph ontology and construction method used in the proposed approach, which organizes information to directly support design-logic-centered retrieval. While GraphRAG also employs a knowledge graph to enhance retrieval, its graph schema and node-level summarization are designed for general-purpose usage. Specifically, GraphRAG tends to construct nodes around project names (*e.g.*, Fallingwater) rather than design logic (*e.g.*, "ways to encourage natural light"). In the dataset used here, the raw materials are already summaries of individual design cases such as Fallingwater; creating project-name nodes therefore duplicates existing information instead of generating concept-level linkages (see examples in Table 5). This structural mismatch reduces the graph's ability to connect related design logic, leading to weaker performance, sometimes even lower than traditional RAG.

4.3. Overall user experience

Fig. 8 shows the percentage of winning responses generated by the proposed method and the GPT-4o across four evaluation dimensions: *comprehensiveness*, *diversity*, *empowerment*, and *directness*. On the left subfigure, default setting was used in the proposed method. On the

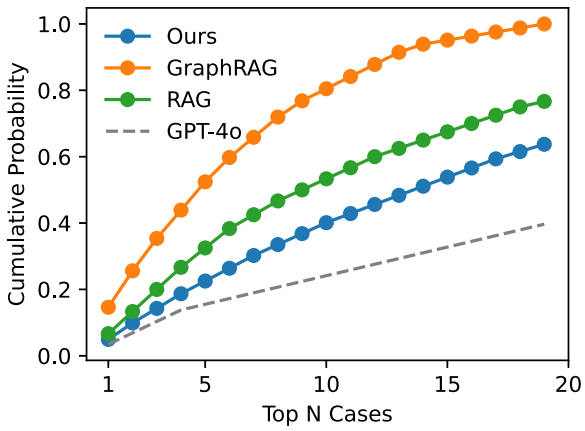


Fig. 7. Cumulative distribution functions of unique design case references across methods.

right, only the text result of the proposed method is presented to the participants in user rating experiments (text-only condition).

As shown in the figure, the results from the user study highlight significant differences between the proposed method and GPT-4o under both default setting and text-only conditions. The proposed method with default setting (left of Fig. 8) significantly outperformed GPT-4o across all evaluated dimensions: comprehensiveness (87.5% vs. 7.8%), diversity (93.8% vs. 0%), empowerment (79.7% vs. 10.9%), and directness (67.2% vs. 25%). The largest advantage was observed in the diversity dimension, where the proposed method achieved an approval rating of 93.8%, highlighting the system's capability to provide varied and rich responses.

In the text-only conditions (right of Fig. 8), two observations are highlighted. First, the critical role of providing targeted images is shown by the clear reductions in the proposed method's metrics. Second, answer quality is examined when images are not attached. Although the proposed method uses a much smaller design-case knowledge base, performance is nearly equivalent to GPT-4o, with only minor differences in empowerment (26.3% proposed method vs. 36.8% GPT-4o) and directness (35.5% proposed method vs. 35.5% GPT-4o). These results indicate that the proposed method can efficiently extract rich design logic from the data and support human designers, even with a small database.

The follow-up interviews provided additional insights. Participants emphasized that image-supported results align more closely with typical search behaviors, particularly among professional architects who appreciated visual content for its ability to enhance engagement and interest. Additionally, the credibility of the system was boosted by the fact that all provided images originated from actual, constructed architectural cases, unlike purely AI-generated images, which are mainly used for inspiration rather than practical implementation. Participants suggested that an ideal system might integrate the authenticity of real-world cases with the creative potential of AI-generated imagery.

Regarding purely textual results, participants noted difficulty distinguishing between real and potentially fabricated examples generated by the GPT-4o. In contrast, the proposed method provides verified real-world examples, reinforcing trust and credibility. Participants also recommended extending the system's functionality to include references to building regulations and detailed architectural construction methods, believing such features would significantly enhance usability.

4.4. Domain adaptations

Based on the performance demonstrated above, the framework can be extended to a broader range of applications. First, it can be adapted for domain-specific architectural design tasks, where users prioritize particular categories of design strategies and objectives. This adaptation is achieved by customizing the LLM system prompts used for design logic extraction and summarization.

Fig. 9 illustrates two examples where the proposed method is applied to kindergarten and passive design respectively. Left column shows input images used in two examples. The top is an interior photo of a kindergarten design [56]. The bottom are two separate images about a school facade with passive design techniques [57]. Middle column includes top 3 extracted design logic using default prompts. Right column contains top 3 or 4 extracted design logic using prompts customized for kindergarten design. Typical analysis for general uses are shown in magenta. Texts referring to the reference material are shown in blue. New insights not covered by the default prompt are shown in green. Potential misinterpretations are shown in orange.

To encourage the LLM to attend to design analysis relevant to the corresponding focuses, a book section on educational building design [58, page 46–49] and another section on passive design [59, page 46–49] are selected as reference materials. The raw text from the sources is extracted, and the LLM is instructed to summarize the embedded design strategies and goals. These extracted elements then replace the default content in the "topic focus" section of the system prompt for design logic extraction (see Section 6.2.1), guiding the LLM to think in the same way as the reference text does.

A noticeable shift in the focus of the LLM outputs is observed when the domain-specific system prompt is used. As depicted in Fig. 9, the resulting analysis becomes more targeted. The kindergarten result (top row) emphasizes the psychological aspect and needs of children, which originates from reference text such as a "sense of refuge", "sense of community", and "collaborative learning" (highlighted in blue). Furthermore, the LLM prioritizes design strategies oriented toward child-centered architecture (e.g., *"open gallery and interconnected balconies"* in green) over more general strategies, such as skylight design, which remains in the default results (middle column in that same Fig. 9). The passive design result (bottom row) shows less shift in extracted content, as the default prompt has included passive design as an analysis topic. Even still, a more targeted analysis can be observed (shown in blue text).

Potential misinterpretations of the case study are also observed. for example, while the inset window areas and vertical chimney are stated to be for ventilation, it is not in a strict sense *cross ventilation*. It shows off-the-shelf LLMs may not be able to handle the corner cases in architecture details, calling for specialized models in future research.

Using the same approach, the framework can also be adapted to align with established design criteria. For instance, users in design education might wish to correlate the AIA (American Institute of Architects) Framework for Design Excellence Principles [60] with broader sets of design cases. While the framework can already perform such mappings, the results can be further improved by tailoring the system prompts using content from the AIA Principles. However, it is important to note that prompt length, and thus computational cost, increases with the size of reference material. To reduce cost, users may employ a basic summarization process as described previously, or apply more advanced LLM-based workflows [61], which fall beyond the scope of this paper.

5. Conclusions

This paper presented a RAG-KG framework tailored to early-stage architectural design, addressing challenges of visual data dominance and linguistic diversity in precedent case archives. By combining multimodal knowledge extraction with a lightweight, domain-specific knowledge graph and a multi-step question-answering pipeline, the proposed approach supports precedent-based design assistance through question answering with design references.

The evaluation shows that the system delivers more coherent and relevant case references compared to prior methods, achieving about

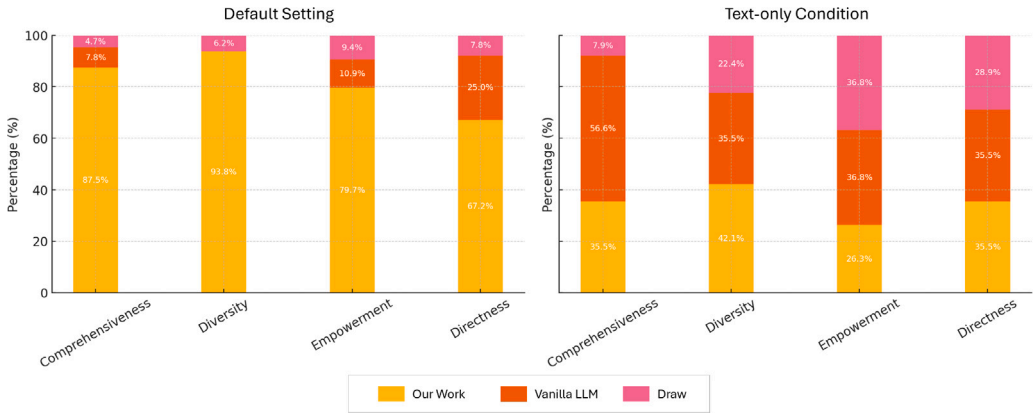


Fig. 8. User rating on question answering quality.

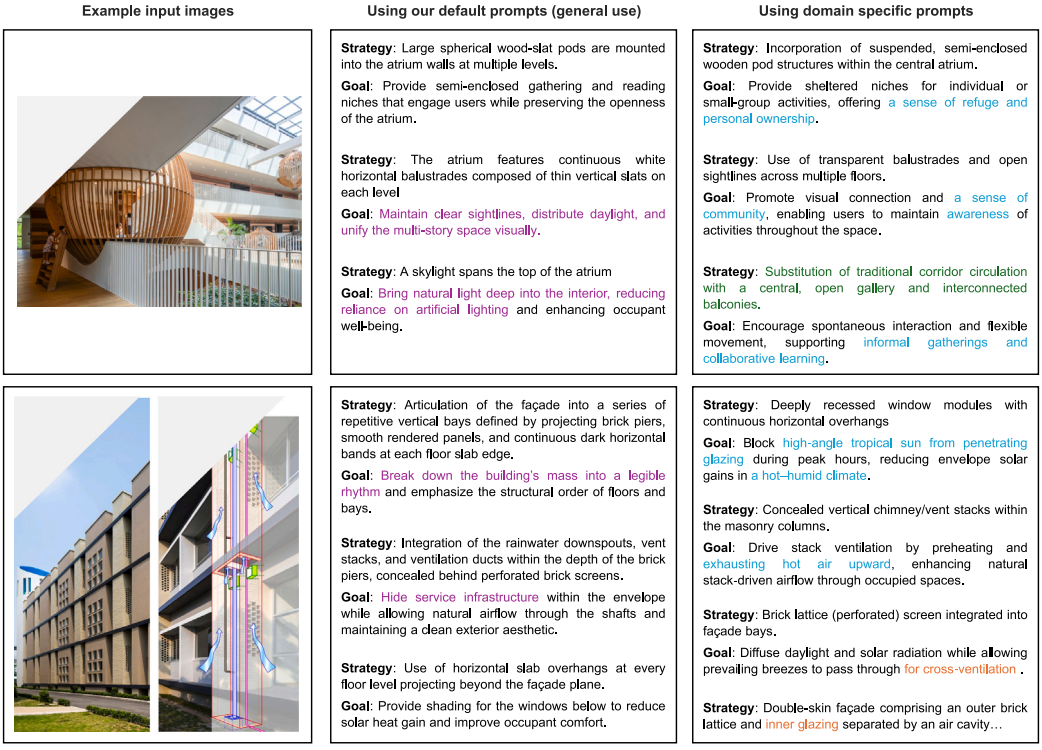


Fig. 9. Customizing knowledge extraction through prompt modification.

20% improvement in reference fitness and 30% improvement in reference diversity, while providing a user experience comparable to off-the-shelf LLMs despite operating on a smaller knowledge base. The performance suggests strong potential to support novice architects and students in early-stage design.

While promising, the framework still faces certain limitations:

- **Advanced design logic representations**: This work focuses on strategy–goal relationships, a subset of design logic that is often explicitly stated in text or evident visually. More complex representations, such as logic chains or conflicting effects (*e.g.*, large windows causing heat loss), are rarely documented in the dataset and may require more advanced reasoning methods, which are left for future work. The proposed method can also be combined with traditional precise-matching query approaches (*e.g.*, Cypher [29]) to further extend its capabilities.
- **Misinterpretations of LLMs in corner cases**: While current LLMs provide robust analysis on common topics such as material usage and design aesthetics, misinterpretations were observed in technical corner cases (see Section 4.4). A more comprehensive evaluation is required when extending the framework to technical analyses such as architectural element details in passive design, structural systems, lighting systems, or HVAC configurations.
- **Lack of detailed architectural data** Publicly available architecture datasets, such as Wikipedia, are dominated by photos and rarely include technical details on structural, lighting, or HVAC systems. This limitation currently constrains the framework's ability to provide guidance on technical systems and detailed architectural components.
- **Limited participant pool in user study** Although the user study included participants with diverse backgrounds, most participants were students, so the conclusions reflect more feedback from novice designers than from experienced designers. Future studies should expand the participant pool to include a larger number and wider range of practicing professionals for broader feedback.

Looking ahead, this work lays the groundwork for expanding precedent-based tools into broader design workflows, with opportunities to further enhance educational use and professional practice. Specifically:

- **Tool integration**: Extend the LLM agent system with features like sketch or screenshot uploads for more reference-grounded suggestions, image generation for visual renderings, and integration with design software (*e.g.*, Rhinoceros) for interactive use.
- **New modalities**: Incorporate metadata such as room adjacency graphs, space syntax, or daylight simulations [7] to support performance-driven reasoning. It remains an open question that how can knowledge from additional modalities (*e.g.*, simulation results) be meaningfully combined with the knowledge extracted from images and texts.
- **Cross-domain use**: Adapt the proposed knowledge-graph approach to other design disciplines such as industrial design, digital storytelling [62], or computer games [63], where visual and linguistic diversity is also prevalent.

CRediT authorship contribution statement

Danrui Li: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Yichao Shi:** Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Data curation. **Mathew Schwartz:** Writing – review & editing, Writing – original draft, Methodology. **Mubbasir Kapadia:** Writing – review & editing, Supervision, Resources, Formal analysis.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT from OpenAI in order to improve the readability and language of the manuscript. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Mubbasir Kapadia reports a relationship with Roblox Corporation that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study has been reviewed by the Institutional Review Board (IRB) at Georgia Institute of Technology (IRB2025-917) and Rutgers, the State University of New Jersey (Pro2025002498).

The research was supported by *SAI: Modeling Equitable and Accessible Public Spaces* (NSF # 2324598). For architectural images that are not licensed under Creative Commons, the corresponding regions are partially masked in the figures to comply with copyright policies.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.autcon.2025.106756.

Data availability

The source code and `WikiArch` dataset with its labels in retrieval benchmark is made public at: https://github.com/danruili/ArchLogicRAG.

References

[1] C. Lee, A. Nettley, M. Prin, P. Owens, How architects use research, 2014, URL: https://www.architecture.com/knowledge-and-resources/resources-landing-page/how-architects-use-research.

[2] B. Lawson, Schemata, gambits and precedent: some factors in design expertise, Des. Stud. 25 (5) (2004) 443–457, http://dx.doi.org/10.1016/j.destud.2004.05.001, Expertise in Design.

[3] A. Yaseen, M. Waqas, A. Mukhtar, Precedent study: An approach to learning about design challenges in architectural studio pedagogy, Glob. Educ. Stud. Rev. VII (I) (2022) 395–403, http://dx.doi.org/10.31703/gesr.2022(VII-I).38.

[4] A. Heylighen, I. Verstijnen, Close encounters of the architectural kind, Des. Stud. 24 (4) (2003) 313–326, http://dx.doi.org/10.1016/S0142-694X(02)00040-6.

[5] C. Schwartz, Investigating the tectonic: Grounding theory in the study of precedents, Int. J. Archit. Spat. Environ. Des. 10 (1) (2015) 1–12, http://dx.doi.org/10.18848/2325-1662/CGP/v10i01/38401.

[6] A. Cavieres, U. Bhatia, P. Joshi, F. Zhao, A. Ram, CBArch: A case-based reasoning framework for conceptual design of commercial buildings, in: AAAI Spring Symposium: Artificial Intelligence and Sustainable Design, AAAI Press, Stanford, CA, USA, 2011, pp. 19–25, URL: https://cdn.aaai.org/ocs/2494/2494-10852-1-PB.pdf.

[7] J.M. Gagne, M. Andersen, L.K. Norford, An interactive expert system for daylighting design exploration, Build. Environ. 46 (11) (2011) 2351–2364, http://dx.doi.org/10.1016/j.buildenv.2011.05.016.

[8] L. Makatura, M. Foshey, B. Wang, F. Hähnlein, P. Ma, B. Deng, et al., How can large language models help humans in design and manufacturing? Part 1: Elements of the LLM-enabled computational design and manufacturing pipeline, Harv. Data Sci. Rev. (Special Issue 5) (2024) http://dx.doi.org/10.1162/99608f92.cc80fe30.

[9] J. Zhang, R. Xiang, Z. Kuang, B. Wang, Y. Li, ArchGPT: harnessing large language models for supporting renovation and conservation of traditional architectural heritage, Herit. Sci. 12 (1) (2024) 220, http://dx.doi.org/10.1186/s40494-024-01334-x.

[10] S. Tu, W. Zhuang, F. Ren, Constructing a knowledge graph for extreme climate architecture based on large language models (LLMs), in: H. Chai, D.W.N. Bao, Z. Guo, P.F. Yuan (Eds.), Symbiotic Intelligence, Springer Nature Singapore, Singapore, 2025, pp. 251–261, http://dx.doi.org/10.1007/978-981-96-3433-0_22.

[11] X. Yang, H. Zhong, Z. Wang, P. Du, K. Zhou, H. Zhou, et al., BEKG: A built environment knowledge graph, Build. Res. Inf. 52 (1–2) (2024) 19–37, http://dx.doi.org/10.1080/09613218.2023.2238851.

[12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, et al., From local to global: A graph RAG approach to query-focused summarization, 2025, URL: https://arxiv.org/abs/2404.16130.

[13] D. Li, Y. Shi, Y. Wang, Z. Shi, M. Kapadia, ArchSeek: Retrieving architectural case studies using vision-language models, 2025, URL: https://arxiv.org/abs/2503.18680.

[14] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, C.D. Manning, RAPTOR: Recursive abstractive processing for tree-organized retrieval, in: International Conference on Learning Representations, ICLR, 2024, URL: https://arxiv.org/abs/2401.18059v1. Paper ID: 2401.18059.

[15] C. Wang, S. Lu, H. Chen, Z. Li, B. Lin, Effectiveness of one-click feedback of building energy efficiency in supporting early-stage architecture design: An experimental study, Build. Environ. 196 (2021) 107780, http://dx.doi.org/10.1016/j.buildenv.2021.107780.

[16] C. Li, H. Pan, A. Bousseau, N.J. Mitra, Sketch2CAD: sequential CAD modeling by sketching in context, ACM Trans. Graph. 39 (6) (2020) http://dx.doi.org/10.1145/3414685.3417807.

[17] F. Kharvari, L.E. Kaiser, Impact of extended reality on architectural education and the design process, Autom. Constr. 141 (2022) 104393, http://dx.doi.org/10.1016/j.autcon.2022.104393.

[18] K.-H. Chang, C.-Y. Cheng, J. Luo, S. Murata, M. Nourbakhsh, Y. Tsuji, BuildingGAN: Graph-conditioned architectural volumetric design generation, in: 2021 IEEE/CVF International Conference on Computer Vision, ICCV, IEEE Computer Society, Los Alamitos, CA, USA, 2021, pp. 11936–11945, http://dx.doi.org/10.1109/ICCV48922.2021.01174.

[19] H. Yan, K. Yan, G. Ji, Optimization and prediction in the early design stage of office buildings using genetic and XGBoost algorithms, Build. Environ. 218 (2022) 109081, http://dx.doi.org/10.1016/j.buildenv.2022.109081.

[20] J. Shin, J.-K. Lee, Indoor Walkability Index: BIM-enabled approach to Quantifying building circulation, Autom. Constr. 106 (2019) 102845, http://dx.doi.org/10.1016/j.autcon.2019.102845.

[21] M. Schwartz, Human centric accessibility graph for environment analysis, Autom. Constr. 127 (2021) 103557, http://dx.doi.org/10.1016/j.autcon.2021.103557.

[22] D. Schaumann, N. Putievsky Pilosof, H. Sopher, J. Yahav, Y.E. Kalay, Simulating multi-agent narratives for pre-occupancy evaluation of architectural designs, Autom. Constr. 106 (2019) 102896, http://dx.doi.org/10.1016/j.autcon.2019.102896.

[23] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Commun. 7 (1) (1994) 39–59, http://dx.doi.org/10.3233/AIC-1994-7104.
[24] T. Narahara, T. Yamasaki, Subjective functionality and comfort prediction for apartment floor plans and its application to intuitive online property search, IEEE Trans. Multimed. 25 (2022) 6729–6742, http://dx.doi.org/10.1109/TMM.2022.3214072.
[25] R. Reinanda, E. Meij, M. de Rijke, Knowledge Graphs: An Information Retrieval Perspective, vol. 14, Now Publishers Inc., Hanover, MA, USA, 2020, pp. 289–444, http://dx.doi.org/10.1561/1500000063.
[26] X. Wang, N. El-Gohary, Deep learning-based relation extraction and knowledge graph-based representation of construction safety requirements, Autom. Constr. 147 (2023) 104696, http://dx.doi.org/10.1016/j.autcon.2022.104696.
[27] Y. Gao, G. Xiong, H. Li, J. Richards, Exploring bridge maintenance knowledge graph by leveraging GrapshSAGE and text encoding, Autom. Constr. 166 (2024) 105634, http://dx.doi.org/10.1016/j.autcon.2024.105634.
[28] C. Zheng, S. Wong, X. Su, Y. Tang, A. Nawaz, M. Kassem, Automating construction contract review using knowledge graph-enhanced large language models, Autom. Constr. 175 (2025) 106179, http://dx.doi.org/10.1016/j.autcon.2025.106179.
[29] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, et al., Cypher: An evolving query language for property graphs, in: Proceedings of the 2018 International Conference on Management of Data, SIGMOD '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 1433–1445, http://dx.doi.org/10.1145/3183713.3190657.
[30] M. Uhm, J. Kim, S. Ahn, H. Jeong, H. Kim, Effectiveness of retrieval augmented generation-based large language models for generating construction safety information, Autom. Constr. 170 (2025) 105926, http://dx.doi.org/10.1016/j.autcon.2024.105926.
[31] S. Baek, C.Y. Park, W. Jung, Automated safety risk management guidance enhanced by retrieval-augmented large language model, Autom. Constr. 176 (2025) 106255, http://dx.doi.org/10.1016/j.autcon.2025.106255.
[32] S. Zhou, X. Liu, D. Li, T. Gu, K. Liu, Y. Yang, et al., Integrating domain-specific knowledge and fine-tuned general-purpose large language models for question-answering in construction engineering management, Autom. Constr. 175 (2025) 106206, http://dx.doi.org/10.1016/j.autcon.2025.106206.
[33] H. Zhang, X. Liu, H. Pan, Y. Song, C.W.-K. Leung, ASER: A large-scale eventuality knowledge graph, in: Proceedings of the Web Conference 2020, WWW '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 201–211, http://dx.doi.org/10.1145/3366423.3380107.
[34] M. Sap, R. Le Bras, E. Allaway, C. Bhagavatula, N. Lourie, H. Rashkin, et al., ATOMIC: an atlas of machine commonsense for if-then reasoning, in: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, in: AAAI'19/IAAI'19/EAAI'19, AAAI Press, 2019, pp. 3027–3035, http://dx.doi.org/10.1609/aaai.v33i01.33013027.
[35] G. Stiny, Introduction to shape and shape grammars, Environ. Plan. B: Plan. Des. 7 (3) (1980) 343–351, http://dx.doi.org/10.1068/b070343.
[36] G. Stiny, Shape: Talking about Seeing and Doing, MIT Press, Cambridge, Massachusetts, 2006, p. 422, http://dx.doi.org/10.7551/mitpress/6201.001.0001.
[37] C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, S. Angel, A Pattern Language: Towns, Buildings, Construction, in: Center for Environmental Structure, vol. 2, Oxford University Press, New York, USA, 1977, http://dx.doi.org/10.2307/1574526.
[38] S. Bisht, K. Shekhawat, N. Upasani, R.N. Jain, R.J. Tiwaskar, C. Hebbar, Transforming an adjacency graph into dimensioned floorplan layouts, in: Computer Graphics Forum, Vol. 41, Wiley Online Library, 2022, pp. 5–22, http://dx.doi.org/10.1111/cgf.14451.
[39] H. Park, H. Suh, J. Kim, S. Choo, Floor plan recommendation system using graph neural network with spatial relationship dataset, J. Build. Eng. 71 (2023) 106378, http://dx.doi.org/10.1016/j.jobe.2023.106378.
[40] F. Pfitzner, S. Hu, A. Braun, A. Borrmann, Y. Fang, Monitoring concrete pouring progress using knowledge graph-enhanced computer vision, Autom. Constr. 174 (2025) 106117, http://dx.doi.org/10.1016/j.autcon.2025.106117.
[41] J. Zhang, X. Ruan, H. Si, X. Wang, Dynamic hazard analysis on construction sites using knowledge graphs integrated with real-time information, Autom. Constr. 170 (2025) 105938, http://dx.doi.org/10.1016/j.autcon.2024.105938.
[42] L. McInnes, J. Healy, N. Saul, L. Grossberger, UMAP: Uniform manifold approximation and projection, J. Open Source Softw. 3 (29) (2018) 861, http://dx.doi.org/10.21105/joss.00861.
[43] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 12 (2011) 2825–2830, http://dx.doi.org/10.5555/1953048.2078195.
[44] J. Liu, LlamaIndex, 2022, http://dx.doi.org/10.5281/zenodo.1234, URL: https://github.com/jerryjliu/llama_index.
[45] G.V. Cormack, C.L.A. Clarke, S. Buettcher, Reciprocal rank fusion outperforms condorcet and individual rank learning methods, in: Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '09, Association for Computing Machinery, New York, NY, USA, 2009, pp. 758–759, http://dx.doi.org/10.1145/1571941.1572114.
[46] W.X. Zhao, J. Liu, R. Ren, J.-R. Wen, Dense text retrieval based on pretrained language models: A survey, ACM Trans. Inf. Syst. 42 (4) (2024) http://dx.doi.org/10.1145/3637870.
[47] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K.V. Alwala, A. Joulin, et al., ImageBind one embedding space to bind them all, in: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2023, pp. 15180–15190, http://dx.doi.org/10.1109/CVPR52729.2023.01457.
[48] T.P. Project, Flask: The Python micro web framework for building web applications, 2010, https://github.com/pallets/flask. First released April 1 2010; (accessed 02 August 2025).
[49] xtekky, ChatGPT interface with better UI, 2024, URL: https://github.com/xtekky/chatgpt-clone, (accessed 03 August 2025).
[50] M. Majlis, Wikipedia-API: A Python wrapper for Wikipedia's API, 2025, https://wikipedia-api.readthedocs.io/en/latest/. Version 0.8.1, (accessed 26 April 2025).
[51] C. Wang, X. Wei, Y. Jiang, F. Ong, K. Gao, X. Yu, et al., Solving the content gap in roblox game recommendations: LLM-based profile generation and reranking, 2025, URL: https://arxiv.org/abs/2502.06802. arXiv:2502.06802.
[52] G. Ye, Nano-graphrag: A lightweight GraphRAG implementation, 2024, URL: https://github.com/gusye1234/nano-graphrag, (accessed 21 January 2025).
[53] M. Mansoury, H. Abdollahpouri, M. Pechenizkiy, B. Mobasher, R. Burke, FairMatch: A graph-based approach for improving aggregate diversity in recommender systems, in: Proceedings of the 28th ACM Conference on User Modeling, Adaptation and Personalization, UMAP '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 154–162, http://dx.doi.org/10.1145/3340631.3394860.
[54] M. Ge, C. Delgado-Battenfeld, D. Jannach, Beyond accuracy: evaluating recommender systems by coverage and serendipity, in: Proceedings of the Fourth ACM Conference on Recommender Systems, RecSys '10, Association for Computing Machinery, New York, NY, USA, 2010, pp. 257–260, http://dx.doi.org/10.1145/1864708.1864761.
[55] A. Jadon, A. Patil, A comprehensive survey of evaluation techniques for recommendation systems, in: A.K. Bairwa, V. Tiwari, S.K. Vishwakarma, M. Tuba, T. Ganokratanaa (Eds.), Computation of Artificial Intelligence and Machine Learning, Springer Nature Switzerland, Cham, 2025, pp. 281–304, http://dx.doi.org/10.1007/978-3-031-71484-9_25.
[56] ArchDaily, Cheer kindergarten / HIBINOSEKKEI + Youji no Shiro, 2025, URL: https://www.archdaily.com/1025442/cheer-kindergarten-hibinosekkei-plus-youji-no-shiro, (Accessed 30 April 2025).
[57] Archdaily, Brick passive designed university / TAISEI design planners architects & engineers, 2017, URL: https://www.archdaily.com/877077/brick-passive-designed-university-taisei-corporation.
[58] M. Dudek, Schools and Kindergartens : A Design Manual, second and revised ed., Birkhäuser, Basel, Switzerland, 2015, http://dx.doi.org/10.1007/978-3-7643-8329-9.
[59] G.Z. Brown, M. DeKay, Sun, Wind, and Light: Architectural Design Strategies, John Wiley & Sons, Incorporated, Somerset, 2014.
[60] American Institute of Architects, AIA framework for design excellence, 2025, URL: https://www.aia.org/design-excellence/aia-framework-design-excellence, (Accessed 18 April 2025).
[61] Artifex Software Inc., PyMuPDF4LLM: PyMuPDF utilities for LLM/RAG, 2025, https://pypi.org/project/pymupdf4llm/. Version 0.0.21, (Accessed 19 April 2025).
[62] D. Li, S.S. Sohn, S. Zhang, C.-J. Chang, M. Kapadia, From words to worlds: Transforming one-line prompts into multi-modal digital stories with LLM agents, in: Proceedings of the 17th ACM SIGGRAPH Conference on Motion, Interaction, and Games, MIG '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 21, http://dx.doi.org/10.1145/3677388.3696321.
[63] D. Li, S. Zhang, S.S. Sohn, K. Hu, M. Usman, M. Kapadia, Cardiverse: Harnessing LLMs for novel card game prototyping, in: C. Christodoulopoulos, T. Chakraborty, C. Rose, V. Peng (Eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Suzhou, China, 2025, pp. 29723–29750, http://dx.doi.org/10.18653/v1/2025.emnlp-main.1511.